CI/CD Pipeline Documentation

> **Complete guide to VeriGraph's automated testing, deployment, and ML model promotion workflows**

Table of Contents

- Quick Visual Overview
- Branch Strategy
- Workflow Overview
- Deployment Targets
- Detailed Workflow Breakdown
- Model Promotion
- Best Practices

Quick Visual Overview

```
┌─────────────────────────────────────────────────────────────────┐
│                          VeriGraph CI/CD Flow                   │
└─────────────────────────────────────────────────────────────────┘

Feature Branch
      │
      │ PR → dev
      ├──────────────► [4 Parallel Test Jobs]
      │                • Backend Unit Tests
      │                • Backend Integration Tests
      │                • Docker Build Test
      │                • Frontend Build
      │
      ▼ (merge)
   dev branch
      │
      │ PR → staging
      ├──────────────► [5 Parallel Validation Jobs]
      │                • Backend Unit Tests
      │                • Backend Integration Tests
      │                • Backend Coverage
      │                • Docker Build + Health Checks
      │                • Frontend (ESLint + Build)
      │
      ▼ (merge)
 staging branch
      │
      │ PUSH (merge)
      ├──────────────► [Deployment Only]
      │                • Build Docker Image
      │                • Deploy to Fly.io Staging
```

```
      │             • URL: https://verigraph.pages.dev/
      │             • Smoke Tests
      │             • Frontend Deployement (configured in Cloudfare
interface)
      │             • URL: https://verigraph-api-staging.fly.dev
      │
      │ PR → main
      ├──────────► [5 Parallel Final Validation Jobs]
      │             • Backend Unit Tests
      │             • Backend Integration Tests
      │             • Backend Coverage
      │             • Docker Build + Health Checks
      │             • Frontend (ESLint + Build)
      │
      ▼ (merge)
  main branch
      │
      │ PUSH (merge)
      ├──────────► [Model Promotion + Deployment]
      │             • Promote Model: Staging → Production
      │             • Archive old Production models
      │             • Build Docker Image
      │             • Deploy to Fly.io Production
      │             • URL: https://verigraph-api.fly.dev
      │             • Smoke
      │             • Frontend Deployement (configured in Cloudfare
interface)
      │             • URL: https://verigraph-8xr.pages.dev/
      │
      ▼
  PRODUCTION
```

Branch Strategy

VeriGraph uses a **three-tier branch strategy** for progressive deployment:

```
dev (development)
  ↓
staging (pre-production)
  ↓
main (production)
```

Branch Purposes

Branch	Purpose	CI/CD Behavior
dev	Active development, feature integration	PRs: Tests only Push: No workflow

Branch	Purpose	CI/CD Behavior
staging	Pre-production testing, QA validation	PRs: Tests + Build validation Push: Deploy to staging
main	Production-ready code	PRs: Final comprehensive tests Push: Model promotion + Deploy to production

Workflow Overview

Summary Table

Event	Branch	Workflow File	Jobs	Deploy	Model Promotion
PR → dev	`dev`	`test.yml`	4 parallel test jobs	No	No
PR → staging	`staging`	`ci-staging.yml`	5 parallel validation jobs	No	No
Push to staging	`staging`	`ci-staging.yml`	Deployment job only	Yes (Staging)	No
PR → main	`main`	`ci-production.yml`	5 parallel final validation jobs	No	No
Push to main	`main`	`ci-production.yml`	Model promotion + Deploy	Yes (Production)	Yes (Staging→Production)

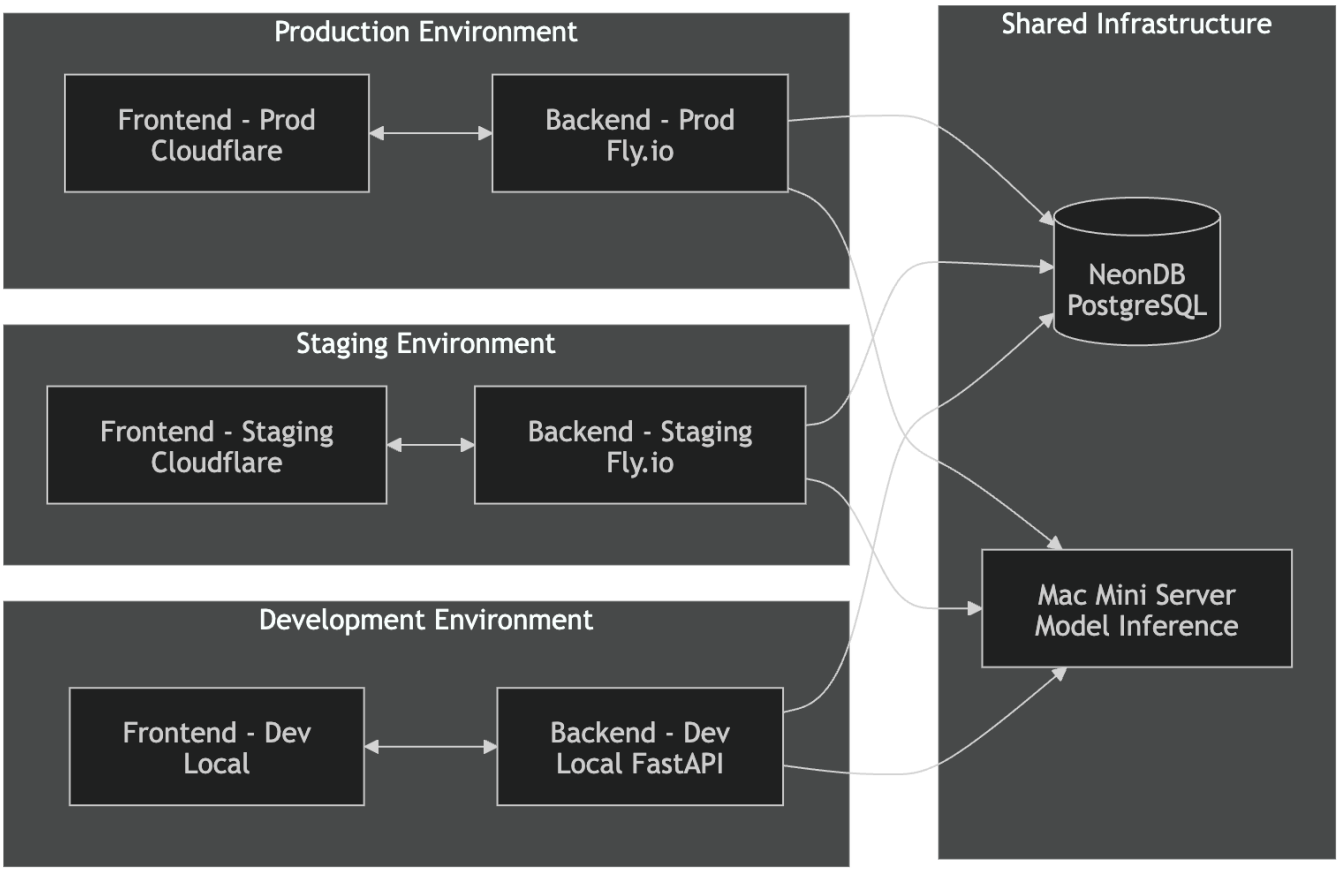
Key Principles

- **Tests run in PRs** - Comprehensive validation before merge
- **Deployments run on push** - After merge, code already validated
- **No redundant tests** - Tests validated in PR don't re-run on push
- **Parallel jobs** - Fast feedback with job separation
- **Clear failure points** - Identify exactly which job failed

Deployment Targets

Environment	URL	Branch	Fly.io App
Staging	https://verigraph-api-staging.fly.dev	`staging`	`verigraph-api-staging`
Production	https://verigraph-api.fly.dev	`main`	`verigraph-api`

Deployment



Detailed Workflow Breakdown

1. Pull Request to `dev` Branch

Workflow: `.github/workflows/test.yml`
Name: `Dev Tests`

Trigger:

```
on:
  pull_request:
    branches:
      - dev
    paths:
      - 'backend/**'
      - '.github/workflows/test.yml'
```

Jobs (4 parallel):

1. **backend-unit-tests**
   - Python 3.11 setup
   - Install dependencies
   - Run unit tests (`pytest -m "not integration"`)
2. **backend-integration-tests**

    - Python 3.11 setup
    - Install dependencies
    - Run integration tests (`pytest -k "test_verify"`)

3. **docker-build**
    - Build Docker image
    - Test container health
    - Verify health endpoint

4. **frontend**
    - Node.js 20 setup
    - Install dependencies (npm ci)
    - Build production bundle

Purpose: Fast feedback for developers. Parallel execution means failure is instantly visible by job name.

---

2. Pull Request to `staging` Branch

Workflow: `.github/workflows/ci-staging.yml`
Name: `Staging`

Trigger:

```yaml
on:
  pull_request:
    branches:
      - staging
    paths:
      - 'backend/**'
      - 'frontend/**'
      - '.github/workflows/ci-staging.yml'
```

Jobs (5 parallel):

1. **backend-unit-tests**
    - Run unit tests with detailed output

2. **backend-integration-tests**
    - Run integration tests
    - Database integration validation

3. **backend-coverage**
    - Full test suite with coverage report
    - Coverage analysis (`pytest --cov`)

4. **docker-build**
    - Build Docker image (`verigraph-api:staging`)
    - Start test container
    - Health check + root endpoint validation
    - Stop and cleanup
5. **frontend**
    - Install dependencies
    - **Run ESLint** (code quality)
    - Build production bundle
    - Verify build artifacts

Purpose: Comprehensive validation before merging to staging. All tests from dev plus coverage and linting.

---

3. Push to `staging` Branch (After Merge)

Workflow: `.github/workflows/ci-staging.yml`
Name: `Staging`

Trigger:

```yaml
on:
  push:
    branches:
      - staging
```

Job:

deploy-backend

1. Build Docker image
2. Setup Fly.io CLI
3. **Deploy to Fly.io** (`fly.staging.toml`)
4. **Smoke tests** (15s wait + endpoint checks)

Why no tests?
Tests already validated in PR. Push only deploys pre-validated code.

Why no model promotion?
Models promoted manually to Staging stage in MLflow UI. Automatic promotion happens only for Production.

Result: Code deployed to staging environment for QA testing. Fast deployment (no redundant tests).

---

4. Pull Request to `main` Branch

Workflow: `.github/workflows/ci-production.yml`
Name: `Production`

Trigger:

```yaml
on:
  pull_request:
    branches:
      - main
    paths:
      - 'backend/**'
      - 'frontend/**'
      - '.github/workflows/ci-production.yml'
```

Jobs (5 parallel):

1. **backend-unit-tests** - Final unit test validation
2. **backend-integration-tests** - Final integration test validation
3. **backend-coverage** - Final coverage check
4. **docker-build** - Final Docker build + health check
5. **frontend** - Final ESLint + build validation

Purpose: Final comprehensive checkpoint before production. Same jobs as staging PR to ensure nothing broke.

---

5. Push to `main` Branch (After Merge) - **PRODUCTION DEPLOYMENT**

Workflow: `.github/workflows/ci-production.yml`
Name: `Production`

Trigger:

```yaml
on:
  push:
    branches:
      - main
```

Jobs (2 sequential):

Job 1: promote-model

Purpose: Promote validated Staging model to Production

Steps:

1. Connect to MLflow (DagHub)
2. Get current Staging model for `fact-checker-gan`

3. Retrieve model metrics (accuracy, F1)
4. Archive old Production models
5. Promote Staging → Production
6. Summary report

Model: `fact-checker-gan` (BERT-GAN Discriminator)

Promotion Logic:

```
# Get Staging model
staging_model = client.get_latest_versions("fact-checker-gan", stages=
["Staging"])[0]

# Archive old Production
old_production = client.get_latest_versions("fact-checker-gan", stages=
["Production"])
for old in old_production:
    client.transition_model_version_stage(old.version, stage="Archived")

# Promote to Production
client.transition_model_version_stage(staging_model.version,
stage="Production")
```

Important: No quality gates at this stage. Model already validated when promoted to Staging manually.

Job 2: deploy-backend (depends on promote-model)

Purpose: Deploy backend to production

Steps:

1. Build Docker image (`verigraph-api:production`)
2. Setup Fly.io CLI
3. **Deploy to Fly.io** (`fly.toml`)
4. **Smoke tests** (15s wait + endpoint checks)
    - Health check: `/health`
    - Root endpoint: `/`

Result:

- Model promoted to Production
- Backend deployed to production
- Production environment validated

Model Promotion

Model Lifecycle

```
┌──────────────┐
│   Training   │  Local/Notebook: Train model with MLflow logging
└──────────────┘
       │
       ▼
┌──────────────┐
│  MLflow Log  │  Log metrics, params, artifacts to DagHub
└──────────────┘
       │
       ▼
┌──────────────┐
│   None       │  New model starts in "None" stage
└──────────────┘
       │
       ▼ MANUAL
┌──────────────┐
│   Staging    │  Manually promote to "Staging" in MLflow UI
└──────────────┘  (after validating metrics)
       │
       ▼ Merge to staging
┌──────────────┐
│  Staging     │  Deploy to staging environment
│  Environment │  Test with staging code
└──────────────┘
       │
       ▼ Merge to main
┌──────────────┐
│  Production  │  AUTO: CI promotes Staging → Production
│  Stage       │  Archives old Production models
└──────────────┘
       │
       ▼
┌──────────────┐
│  Production  │  Model deployed with production code
│  Environment │
└──────────────┘
```

Tracked Model

Model Name	Type	Promotion
`fact-checker-gan`	BERT-GAN Discriminator	Staging → Production (automatic)

Note: `fact-checker-bert` model removed from CI/CD (no longer tracked).

Model Promotion Best Practices

1. **Train model** → Log to MLflow with comprehensive metrics
2. **Validate metrics** → Review accuracy, F1, precision, recall
3. **Manual staging promotion** → In MLflow UI, promote to "Staging" stage
4. **Deploy to staging** → Merge to staging branch, test deployment

5. **Validate in staging** → Test model predictions with staging API
6. **Merge to main** → CI automatically promotes Staging → Production
7. **Verify in production** → Check production API uses new model

---

Required Secrets

Configure these in GitHub repository settings → Secrets and variables → Actions:

Secret	Purpose	Used In
`NEON_DB_URL`	PostgreSQL database connection	All test workflows
`DAGSHUB_REPO`	DagHub repository (e.g., `owner/repo`)	Staging deploy, Production deploy
`DAGSHUB_USER`	DagHub username	Staging deploy, Production deploy
`DAGSHUB_TOKEN`	DagHub API token	Staging deploy, Production deploy
`FLY_API_TOKEN`	Fly.io deployment token	Staging deploy, Production deploy

---

Local Testing

Test Backend Locally

```
cd backend
# Activate virtual environment
source venv/bin/activate  # or `venv\Scripts\activate` on Windows

# Run unit tests
pytest test_main.py -v -m "not integration"

# Run integration tests
pytest test_main.py -v -k "test_verify"

# Run with coverage
pytest test_main.py -v --cov=. --cov-report=term-missing
```

Test Frontend Locally

```
cd frontend

# Install dependencies
npm ci

# Run linter
npm run lint

# Build production bundle
npm run build
```

```
# Check build output
ls -lh dist/
```

Test Docker Locally

```
cd backend

# Build image
docker build -t verigraph-api:local .

# Run container
docker run -d --name test-api -p 8080:8080 -e TEST_MODE=true verigraph-
api:local

# Test endpoints
curl http://localhost:8080/health
curl http://localhost:8080/

# Cleanup
docker stop test-api && docker rm test-api
```

Test Model Promotion Locally

```
cd backend
python scripts/test_promotion_local.py
```

This script will:

1. List all model versions (Production, Staging, Archived, None)
2. Show Staging model metrics
3. Show current Production model
4. Prompt for promotion confirmation
5. Execute promotion with archive of old Production version

Best Practices

Development Workflow

```
# 1. Create feature branch from dev
git checkout dev
git pull origin dev
git checkout -b feature/my-feature

# 2. Make changes, commit
```

```
git add .
git commit -m "feat: add new feature"

# 3. Push and create PR to dev
git push origin feature/my-feature
# Create PR on GitHub: feature/my-feature → dev

# 4. Wait for CI (4 parallel test jobs)
# 5. Get approval, merge to dev
```

Staging Deployment Workflow

```
# 1. On dev branch, create PR to staging
git checkout dev
# Create PR on GitHub: dev → staging

# 2. Wait for CI (5 parallel validation jobs)
# 3. Get approval, merge to staging

# 4. Automatic deployment to staging
# Check: https://verigraph-api-staging.fly.dev/health

# 5. QA testing on staging environment
```

Production Deployment Workflow

```
# 1. Ensure Staging model is ready
# - Check MLflow UI: model in "Staging" stage with good metrics
# - Test model predictions on staging API

# 2. Create PR from staging to main
# Create PR on GitHub: staging → main

# 3. Wait for CI (5 parallel final validation jobs)

# 4. Get approval, merge to main

# 5. Automatic:
#    - Model promotion: Staging → Production
#    - Deployment to production
#    - Check: https://verigraph-api.fly.dev/health
```

Before Merging to main:

- **All tests pass on staging**
- **Staging deployment successful**
- **Model validated in Staging stage** (MLflow UI)

- **QA testing completed on staging**
- **No breaking changes**

Monitoring & Debugging

Check Deployment Status

Staging:

```
curl https://verigraph-api-staging.fly.dev/health
```

Production:

```
curl https://verigraph-api.fly.dev/health
```

View Model Status

DagHub/MLflow UI:

- URL: https://dagshub.com/MarcoSrhl/NLP-Fact-checking
- Navigate to "Models" → "fact-checker-gan"
- View stages: Production, Staging, Archived, None

CI/CD Logs

GitHub Actions:

- Repository → Actions tab
- Select workflow run
- View logs for each parallel job

Identify failures quickly:

- Job names indicate what failed (e.g., "backend-unit-tests")
- Click job to see detailed logs
- Parallel jobs mean only failed component shows red

Quick Reference Guide

Common Scenarios

"I want to test my changes"

```
# Create feature branch and PR to dev
git checkout -b feature/my-feature
```

```
# ... make changes ...
git push origin feature/my-feature
# Create PR to dev → 4 parallel test jobs run
```

"I want to deploy to staging"

```
# After dev PR merged:
# Create PR: dev → staging (5 parallel validation jobs)
# After PR merged → automatic staging deployment
# Check: https://verigraph-api-staging.fly.dev
```

"I want to deploy to production"

```
# After staging tested:
# Create PR: staging → main (5 parallel final validation jobs)
# After PR merged → automatic model promotion + production deployment
# Check: https://verigraph-api.fly.dev
```

"How do I promote a new model?"

```
# 1. Train model, log to MLflow
# 2. Check metrics in DagHub UI
# 3. Manually promote to "Staging" stage in MLflow UI
# 4. Deploy code to staging (merge to staging branch)
# 5. Test model on staging API
# 6. Merge to main → automatic Production promotion
```

"A test failed in CI - how do I debug?"

```
# 1. Check GitHub Actions logs
# 2. Job name tells you what failed (e.g., "backend-coverage")
# 3. Run same test locally:
cd backend
pytest test_main.py -v --cov=. --cov-report=term-missing

# 4. Fix issue, commit, push
# 5. CI re-runs automatically
```

Last Updated: March 3, 2026
Maintained By: VeriGraph Team